

NSIGHT CI/CD 운영 매뉴얼

GitLab + GitLab Runner + Gradle + 17 WAR 배포 기준

문서 정보

항목	내용
시스템	NSIGHT 마케팅 플랫폼
형상관리	GitLab
CI/CD	GitLab CI/CD
Build Tool	Gradle
Runner	GitLab Runner
배포대상	17 WAR
WAS	Apache Tomcat
Session	Spring Session JDBC
DB	RDW / ADW

제1장 목적

본 문서는 NSIGHT 마케팅 플랫폼의 CI/CD 운영 기준을 정의한다.

목표는 다음과 같다.

1. 개발 표준화
 2. 형상관리 표준화
 3. 빌드 자동화
 4. 배포 자동화
 5. 롤백 자동화
 6. 운영 안정성 확보
-

제2장 CI/CD 아키텍처

전체 구성

개발자

↓

GitLab

↓

Merge Request

↓

GitLab Runner

↓

Gradle Build

↓

Unit Test

↓

SonarQube

↓

Nexus

↓

WAR 생성

↓

배포서버

↓

Tomcat

↓

Health Check

↓

서비스 오픈

제3장 GitLab Repository 구조

권장 구조

```
nsight-platform  
  
├ common  
├ common-api  
├ common-core  
  
├ cc-service  
├ ic-service  
├ pc-service  
├ bc-service  
  
├ ms-service  
├ sv-service  
  
├ pd-service  
├ cm-service  
├ eb-service  
├ ep-service  
  
├ bp-service  
├ bd-service  
  
├ ss-service  
├ cs-service  
├ ct-service  
  
├ mg-service  
  
└ deploy
```

제4장 Branch 전략

Main

운영

```
main
```

Develop

통합개발

develop

Release

배포 준비

release/1.0.0

Feature

기능 개발

feature/CC-1001
feature/CM-2001

Hotfix

긴급 수정

hotfix/CC-9001

제5장 GitLab Runner 운영

구성

Runner-01

CI Build

Runner-02

배포

Executor

권장

Shell Executor

또는

Docker Executor

Tag

tags:
- build

tags:
- deploy

제6장 Gradle 운영

표준 버전

Gradle 8.x
JDK 21

Build

./gradlew clean build

WAR 생성

./gradlew bootWar

결과

```
build/libs/*.war
```

제7장 GitLab CI Pipeline

단계

```
build
↓
test
↓
quality
↓
package
↓
deploy
↓
health-check
```

제8장 .gitlab-ci.yml 표준

Build

```
build:
  stage: build
```

```
script:  
- ./gradlew clean compileJava
```

Test

```
test:  
stage: test  
  
script:  
- ./gradlew test
```

Package

```
package:  
stage: package  
  
script:  
- ./gradlew bootWar
```

Artifact

```
artifacts:  
paths:  
- build/libs/*.war
```

제9장 품질 검증

SonarQube

필수

검증 항목

Bug

0

Critical

0

Coverage

80%

이상

Security

High

0건

제10장 배포 전략

방식

Rolling Deployment

절차

Tomcat-01

배포

↓

Health Check

↓

Tomcat-02

배포

↓

Health Check

효과

무종단

제11장 WAR 배포 기준

배포 단위

WAR

배포 예

cc.war

cm.war

mg.war

금지

Class 단위 배포

JSP 단위 수동 배포

운영 서버 직접 수정

제12장 Health Check

URL

/actuator/health

성공

```
{  
  "status": "UP"  
}
```

실패

배포 중단

자동 롤백

제13장 롤백 전략

기준

Health Check 실패

절차

이전 WAR

복구

↓

Tomcat Restart

↓

Health Check

제14장 배포 승인

개발

자동

검증

승인 필요

운영

Change 승인 필수

제15장 Artifact 관리

저장소

Nexus

보관

90일

형식

cc-1.0.0.war

cm-1.0.0.war

제16장 운영 점검

GitLab

- Pipeline 상태
 - Runner 상태
-

Runner

- CPU
 - Memory
 - Disk
-

Nexus

- 용량
 - Artifact 수
-

제17장 장애 대응

Case 1

Runner Down

조치

Runner 재기동

Case 2

Gradle Build 실패

조치

Compile Error 확인

Case 3

Test 실패

조치

JUnit 확인

Case 4

배포 실패

조치

Health Check 확인

Case 5

롤백 실패

조치

이전 WAR 수동 복구

제18장 DR 운영

GitLab

DR GitLab 운영

Nexus

이중화

Runner

이중화

배포

DR 센터 동일 절차

제19장 운영 KPI

항목	목표
Build Success	95% 이상
Deploy Success	99% 이상
Rollback Success	100%
Coverage	80% 이상
Critical Bug	0
Security High	0
Availability	99.99%

부록 A 표준 Pipeline

Commit

↓

Build

↓

Test

↓

Sonar

↓

Package

↓

Nexus

↓

Deploy

↓

Health Check

↓

Open

부록 B 운영 체크리스트

☐ GitLab 상태 확인

☐ Runner 상태 확인

☐ Gradle Build 확인

☐ Sonar 결과 확인

☐ Nexus 확인

☐ WAR 생성 확인

☐ Health Check 확인

☐ Rollback 테스트 확인

☐ 운영 승인 확인

☐ 배포 결과 확인

NSIGHT CI/CD 운영 철학

배포는 빠르게 해야 한다.

그러나 운영은 안전해야 한다.

CI/CD의 목적은 배포 자동화가 아니라 운영 가능한 품질을 반복 가능하게 만드는 것이다.

개발자가 만든 소스가 동일한 절차로 빌드되고, 동일한 절차로 검증되고, 동일한 절차로 운영에 반영될 수 있어야 한다.

이것이 NSIGHT CI/CD의 목표이다.